

Lab-5 2A

Jibon Naher¹ and Samnun Azfar²

¹Lecturer, CSE

²Junior Lecturer, CSE

February 26, 2026

Part A (Mutex): Priority Logging System

Problem Statement

Concurrent threads print High or low priority log messages. High-priority messages must always be processed/printed before Low-priority ones, regardless of arrival time within the critical window.

Input (Thread Actions)

- Thread 1 (Low): Prints "Network OK"
- Thread 2 (High): Prints "Disk Failure"
- Thread 3 (Low): Prints "User Login"
- Thread 4 (High): Prints "Power Surge"

Constraints

- Use **one mutex**.

Expected Output (Buffer Dump)

```
[HIGH] Disk Failure
[HIGH] Power Surge
[LOW] Network OK
[LOW] User Login
```

Part B (Semaphore): Ordered Startup System

Problem Statement

Three subsystems (Power, Communication, Control) must initialize in strict order.

Input

- Threads for Power, Comm, and Control start in random order (e.g., Control starts first, then Comm, then Power).

Constraints

- Use semaphores as signals. No mutexes.

Expected Output

```
[Control Thread] Waiting for Signal...
[Comm Thread] Waiting for Signal...
[Power Thread] Power Initialized. Signal sent.
[Comm Thread] Communication Initialized. Signal sent.
[Control Thread] Control System Initialized.
```

Delay Functions

To simulate lengthy service times in concurrent threads, explicit delay functions are introduced so that a thread occupies the server for a realistic duration before releasing it.

On Linux systems, sleep-based system calls are commonly used. The required headers and examples are shown below.

```
#include <unistd.h>    // sleep(), usleep()
#include <time.h>       // nanosleep()
```

Basic delays can be implemented as:

```
sleep(2);              // delay for 2 seconds
usleep(500000);        // delay for 500 milliseconds
```

For higher precision, `nanosleep()` provides finer control:

```
struct timespec ts;
ts.tv_sec = 1;
ts.tv_nsec = 200000000; // 200 ms
nanosleep(&ts, NULL);
```

These delay mechanisms enable realistic modeling of service duration, contention, and waiting behavior in multi-threaded simulations.